

Leçon 5 : Les Sous Programmes en Langage Pascal

UVCI

Table des matières



I - 1- Les procédures	3
II - Application 1 :	5
III - II- Les fonctions	7
IV - Application 2 :	9

1- Les procédures

I

Définition : 1.1 Définition

Une procédure est un sous programme qui retourne un ou plusieurs résultats de différents types.

Syntaxe : 1.2 Déclaration d'une procédure

Procedure nom_proc(liste de paramètres) ;

Var

Nom_variable : type ;

Begin

Instruction(s) ;

end;

Après le nom de la procédure, il faut donner la liste des paramètres (s'il y en a) avec leurs types respectifs. Ces paramètres sont appelés paramètres formels.

Remarque : Remarque :

Les paramètres sont facultatifs, mais s'il n'y en a pas, on met simplement le nom de la procédure.

1.3 L'appel d'une procédure

Pour déclencher l'exécution d'une procédure dans un programme, il suffit de l'appeler. L'appel de procédure s'écrit en mettant le nom de la procédure, puis la liste des paramètres, séparés par des virgules.

A l'appel d'une procédure, le programme interrompt son déroulement normal, exécute les instructions de la procédure, puis retourne à au programme appelant et exécute l'instruction suivante.

Syntaxe : Syntaxe

Nom_procedure(liste de paramètres); ou *Nom_procedure*;

Remarque

Les paramètres utilisés lors de l'appel d'une procédure sont appelés *paramètres effectifs*.

Ces paramètres donneront leurs valeurs aux *paramètres formels*.

Exemple : Exemple :

Énoncé :

Écrire un programme qui affiche à l'écran une ligne de 10 arobase puis passe à la ligne suivante.

Solution :

```
program sousprom1 ;
```

```
uses crt ;
```

```
//***** Création de la procédure *****
```

```
Procedure arobase ; // nom de la procédure ici nous n'avons aucun paramètre
```

```
Var
```

```
i : integer ; // la variable qui nous permettra de faire la variation de la répétition
```

```
begin
```

```
for i := 1 to 10 do // permet de répéter 10 fois
```

```
begin
```

```
write('@') ; // affichera @@@@ @@@@ @@@@
```

```
end ;
```

```
end ;
```

```
begin
```

```
//***** Appel de la procédure dans le programme principal
```

```
*****
```

```
arobase ;
```

```
end.
```



Remarque : Remarque :

Pour permettre à une procédure ou une fonction de modifier le contenu d'une variable passée en paramètre on utilise le mot réservé *Var*.

Application 1 :


II

Exercice

Donner la syntaxe de déclaration correcte :

- ☐ Procedure nom_proc(liste de paramètres) : type;VarNom_variable : type ;BeginInstruction(s) ;end;
- ☐ function nom_proc(liste de paramètres) ;VarNom_variable : type ;BeginInstruction(s) ;end;
- ☐ Procedure nom_proc(liste de paramètres) ;VarNom_variable : type ;BeginInstruction(s) ;end;

Exercice

Énoncé :

Écrire un programme qui permet de calculer le périmètre et l'aire d'un rectangle.

TAF : Veuillez remplir toutes les trous avec les valeurs qui conviennent.

NB : Toutes les réponses ne doivent pas avoir des espaces et elles doivent être en minuscule .

Solution :

```

    perimetre
    aire

uses crt;

var
    longueur, largeur, perimetre, aire : real;

procedure calcul(longueur, largeur : real);
begin
    writeln('Entrer la longueur :');
    readln(longueur);
    writeln('Entrer la largeur :');
    readln(largeur);
end;

procedure recupdonne(longueur, largeur : real);
begin
    peri := longueur * 2 + largeur * 2;
    aire := longueur * largeur;
end;

procedure affiche(longueur, largeur, perimetre, aire : real);
begin
    writeln('le périmètre est : (' , longueur, ' + ' , largeur, ') * 2 =', peri);
    writeln('Aire est : ' , longueur, ' * ' , largeur, '= ', aire);
end;

begin
    recupdonne(longueur, largeur);
    calcul(longueur, largeur, perimetre, aire);
    affiche(longueur, largeur, perimetre, aire);
    readkey;
end.

```

II- Les fonctions



Définition : 2.1 Définition

La fonction est un sous-programme admettant des paramètres et retournant un seul résultat et un seul de type simple qui peut apparaître dans une expression, dans une comparaison, à la droite d'une affectation, etc.

Syntaxe : 2.2 Déclaration d'une procédure

Function nom_Fonct (liste de paramètres) : type;

Var

Nom_variable : type ;

begin

Instruction(s) ;

nom_Fonct := Expression ;

end ;

La syntaxe de la déclaration d'une fonction est assez proche de celle d'une procédure à laquelle on ajoute un type qui représente le type de la valeur retournée par la fonction à partir de *du nom de la fonction* qui reçoit l'expression.

Cette dernière instruction renvoie au programme appelant le résultat de l'expression placée à la suite *du nom de la fonction*.

Remarque : Remarque :

Les paramètres sont facultatifs, mais s'il n'y pas de paramètres, on met simplement le nom de la fonction suivi de son type.

2.3 L'appel d'une fonction

Pour exécuter une fonction, il suffit de faire appel à elle en écrivant son nom suivi des paramètres effectifs. C'est la même syntaxe qu'une procédure.

A la différence d'une procédure, la fonction retourne une valeur. L'appel d'une fonction pourra donc être utilisé dans une instruction (affichage, affectation, ...) qui utilise sa valeur.

Syntaxe : Syntaxe

VARIABLE := *Nom_Fonc*(list de paramètres) ; ou *VARIABLE* := *Nom_Fonc*;

Exemple : Exemple :

Énoncé :

Définir une fonction qui renvoie le plus grand de deux nombres différents.

Solution :

```
program maxi ;
```

```
uses crt;
```

```
var
```

```
grd,nb1,nb2 : integer ;
```

```
//***** Création de la fonction *****
```

Function Max(X: integer , Y: integer) : integer ;*//nom de la fonction avec en paramètre les deux variables contenant chacune une valeur*

```
begin
```

```
if X > Y then // vérifie laquelle des variables et la plus grande
```

```
begin
```

```
Max := X ; // renvoie le X dans ce cas et sort de la fonction
```

```
end
```

```
else
```

```
begin
```

```
Max :=Y ; // renvoie le Y dans ce cas et sort de la fonction
```

```
end ;
```

```
end ;
```

```
//***** Programme principal *****
```

```
begin
```

```
writeln('Entrer le premier nombre');
```

```
readln(nb1);
```

```
writeln('Entrer le deuxième nombre') ;
```

```
readln(nb2) ;
```

```
grd :=Max(nb1,nb2) ; //Appel de ma fonction
```

```
writeln('La plus grande valeur est : ',grd) ;
```

```
end ;
```


Application 2 :


IV

Exercice

Donner la syntaxe de déclaration correcte :

☐ Function nom_Fonct (liste de paramètres) : type;VarNom_variable : type ;beginInstruction(s) ;

nom_Fonct := Expression ;end ;

☐ procedure nom_Fonct (liste de paramètres) : type;VarNom_variable : type ;beginInstruction(s) ;

nom_Fonct := Expression ;end ;

☐ Function nom_Fonct (liste de paramètres) ;VarNom_variable : type ;beginInstruction(s) ;

variable := Expression ;end ;

Exercice

Énoncé :

Écrire un programme qui permet de calculer le périmètre et l'aire d'un rectangle.

TAF : Veuillez remplir toutes les trous avec les valeurs qui conviennent.

NB : Toutes les réponses ne doivent pas avoir des espaces et elles doivent être en minuscule .

Solution :

```

[ ] periaire [ ]

uses crt;

var
[ ] ,largeur, [ ] ,aire : [ ] ;

procedure [ ] ( [ ] , [ ] :real);
[ ]

writeln('Entrer la longueur :');
readln(long);
writeln('Entrer la largeur :');
readln(larg);
end;

[ ] [ ] ( [ ] , [ ] : real) :real;

begin
[ ] := [ ]

end;

[ ] [ ] ( [ ] , [ ] : real) :real;

begin
[ ] := [ ]

end;

procedure [ ] ( [ ] , [ ] , [ ] , [ ] : [ ] );

begin
writeln('le périmètre est : (' ,long,' + ' ,larg,') * 2 =' ,peri);
writeln('Aire est : ' ,long,' * ' ,larg, '=' ,aire);
[ ]

begin
recupdonne(longueur,largeur);
perimetre [ ] calculperi(longueur,largeur);
aire [ ] calculaire(longueur,largeur);

```

```
afficheresul(longueur,largeur,perimetre,aire);  
readkey;  
end.
```